AULA 4

Aumentando seu repertório

Recapitulando ...

Na última aula, aprendemos a consultar dados estruturados usando DuckDB. Criamos um mini DW com tabelas de vendas, produtos e clientes, e vimos como fazer queries SQL para responder perguntas de negócio. Isso foi ótimo para dados que "moram" em tabelas, como números, datas e categorias.

Mas e quando os dados são textos? Documentos, manuais, artigos, *e-mails*, contratos, transcrições de reuniões?

PRA REFLETIR

Pense no seguinte cenário: você trabalha em uma empresa que tem 10.000 documentos internos, como políticas de RH, manuais técnicos, atas de reunião, relatórios de projeto. Um funcionário novo pergunta: "Qual é a política de *home office* da empresa?". Como você encontraria o documento certo?

Se fosse usar busca tradicional por palavras-chave, você teria que torcer para que o documento contenha exatamente os termos "política" e "*home office*". Mas e se o documento usar "trabalho remoto"? Ou "teletrabalho"? Ou "trabalho híbrido"? A busca tradicional não encontraria nada.

Portanto, para resolver esse problema, precisamos conversar sobre *embeddings* e busca semântica. Esses conceitos são a base do RAG (*retrieval-augmented generation*) que implementaremos na próxima aula, então preste bastante atenção.

EMBEDDINGS

Computadores são máquinas de calcular. Por trás de toda a interface bonita, dos *emojis* e das fotos de gatos, existe apenas uma coisa: números. Zeros e uns. Operações matemáticas. Somas, multiplicações, comparações.

Quando você digita "gato" no teclado, o computador não "vê" um gato. Ele vê uma sequência de códigos: 103, 97, 116, 111 (os códigos ASCII das letras g, a, t, o). Para o computador, "gato" é tão diferente de "felino" quanto é de "xyzw123". São apenas sequências diferentes de números para ele.

Então, como fazer um computador "entender" que "gato" e "felino" significam coisas parecidas? Como fazer ele perceber que "automóvel" e "carro" são praticamente sinônimos? Como fazer ele compreender que "saudade" em português está relacionado com "*longing*" em inglês?

A resposta está nos ***embeddings***.

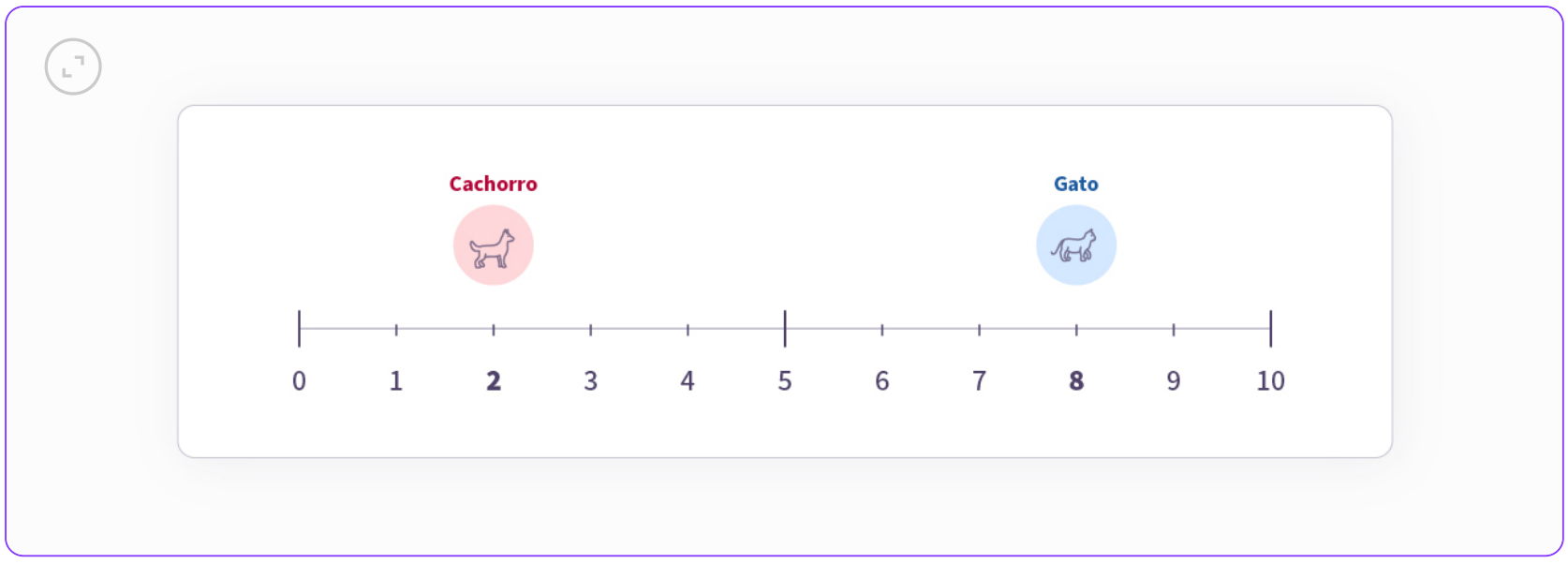
Um *embedding* é uma forma de representar palavras, frases ou documentos como vetores numéricos (ou seja, listas de números). Esses números são calculados de forma que conceitos similares tenham números similares.

Vamos construir essa ideia passo a passo com uma analogia.

Imagine que você quer criar um "mapa" onde cada animal tem uma posição. Você decide usar uma régua simples, de 0 a 10:

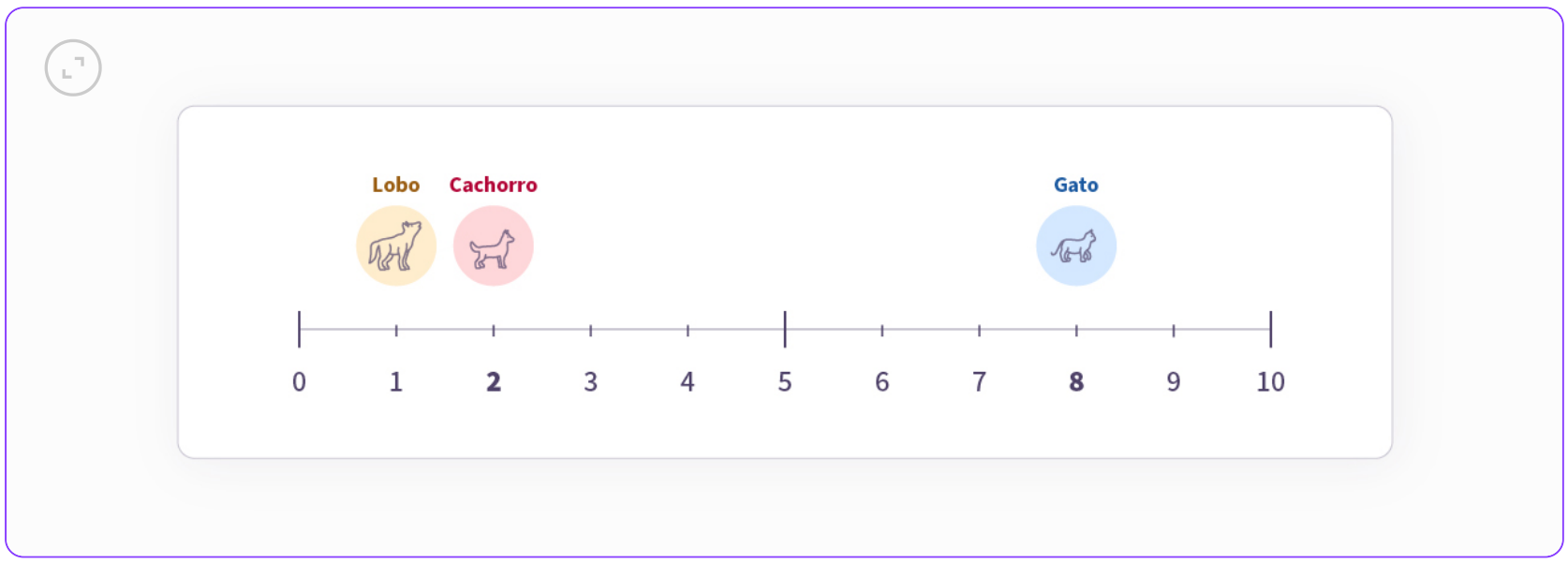
0 ---------- 5 ---------- 10

Você coloca "cachorro" na posição 2 e "gato" na posição 8:



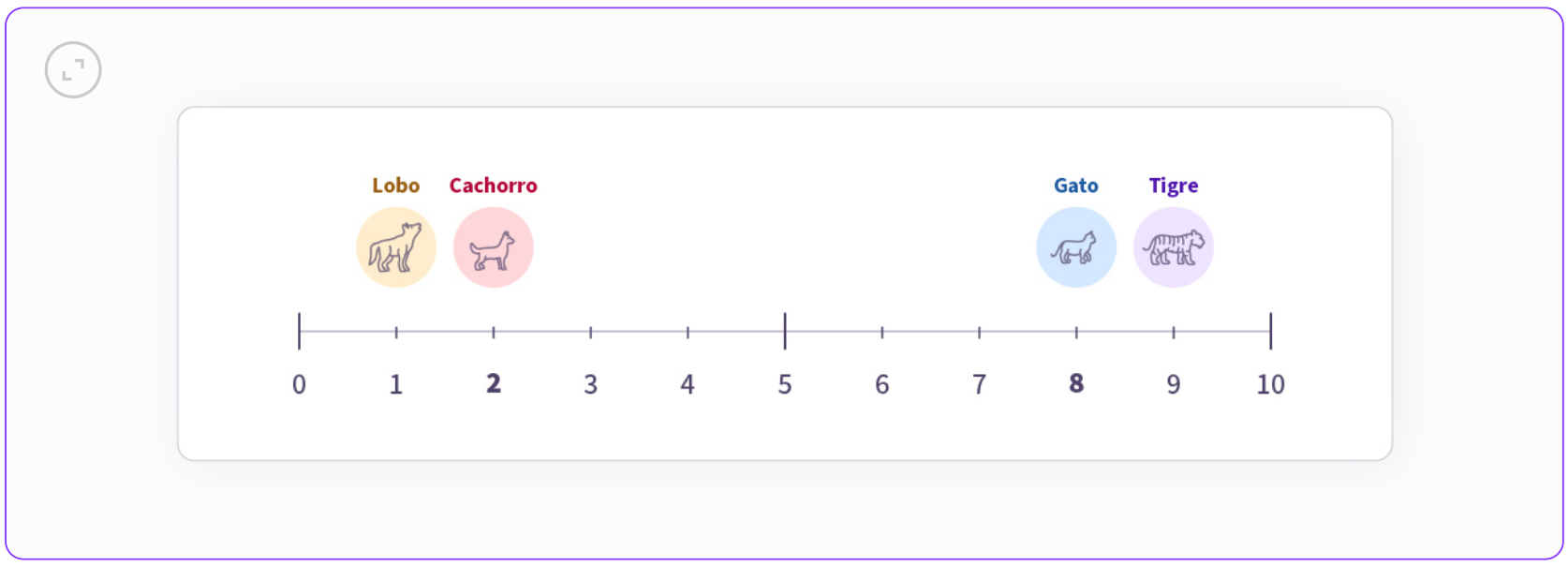
PUCPR Online.

Agora, onde colocar outros animais? Um lobo é parecido com cachorro (os dois são canídeos), então faz sentido colocá-lo perto:



PUCPR Online.

E um tigre? É um felino, como o gato, então deveria ficar perto do gato:



PUCPR Online.

Percebe a lógica? **Animais parecidos ficam próximos na régua. A "distância" entre os números representa a "distância" entre os conceitos.**

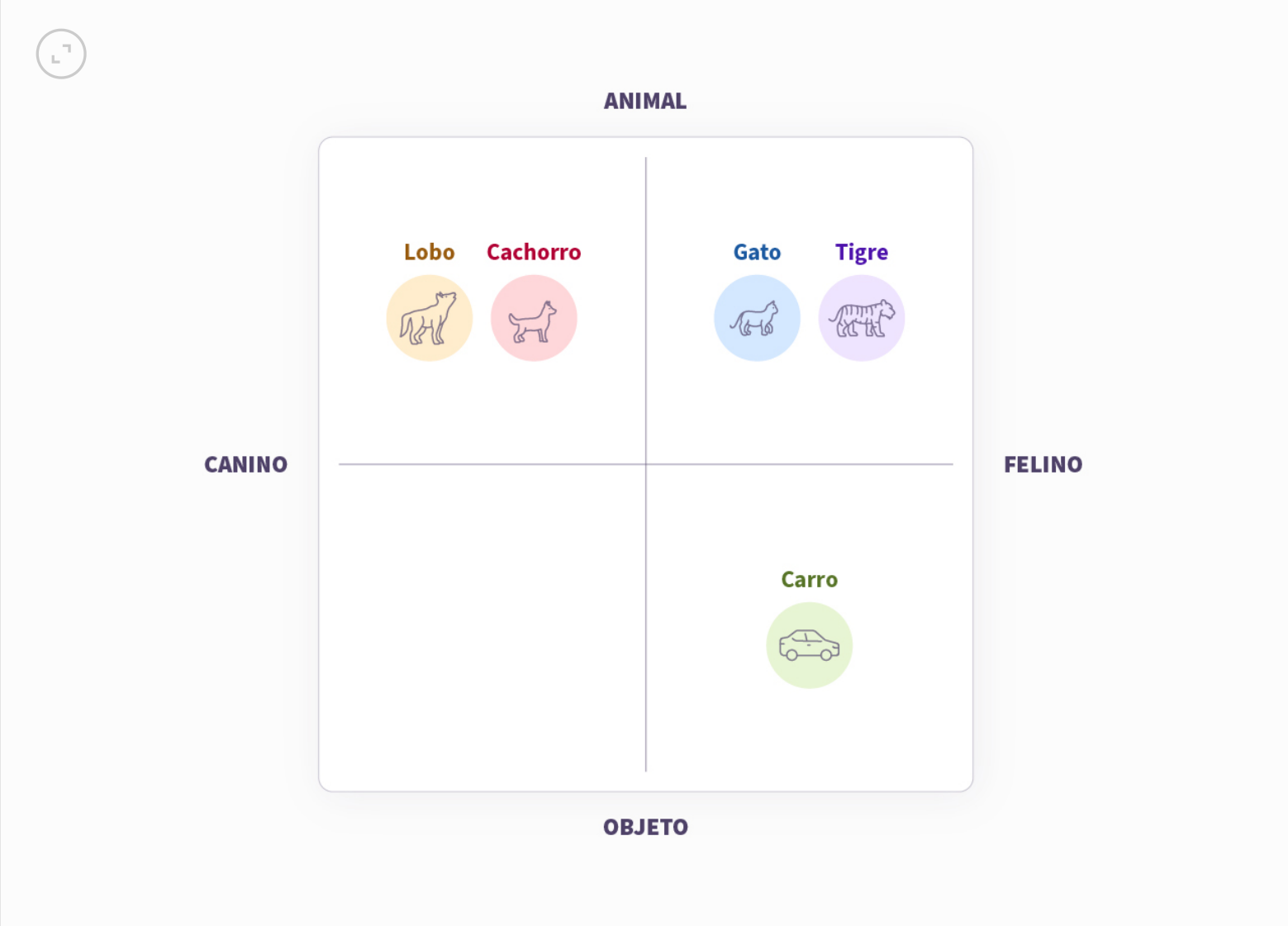
Agora, pensemos em um outro caso: onde colocaríamos um carro nessa régua? 😬

Um carro não é nem parecido com cachorro, nem com gato. Não faz sentido colocá-lo em nenhum ponto dessa linha. Precisamos de uma segunda dimensão (um segundo eixo).

Imagine agora um plano com dois eixos:

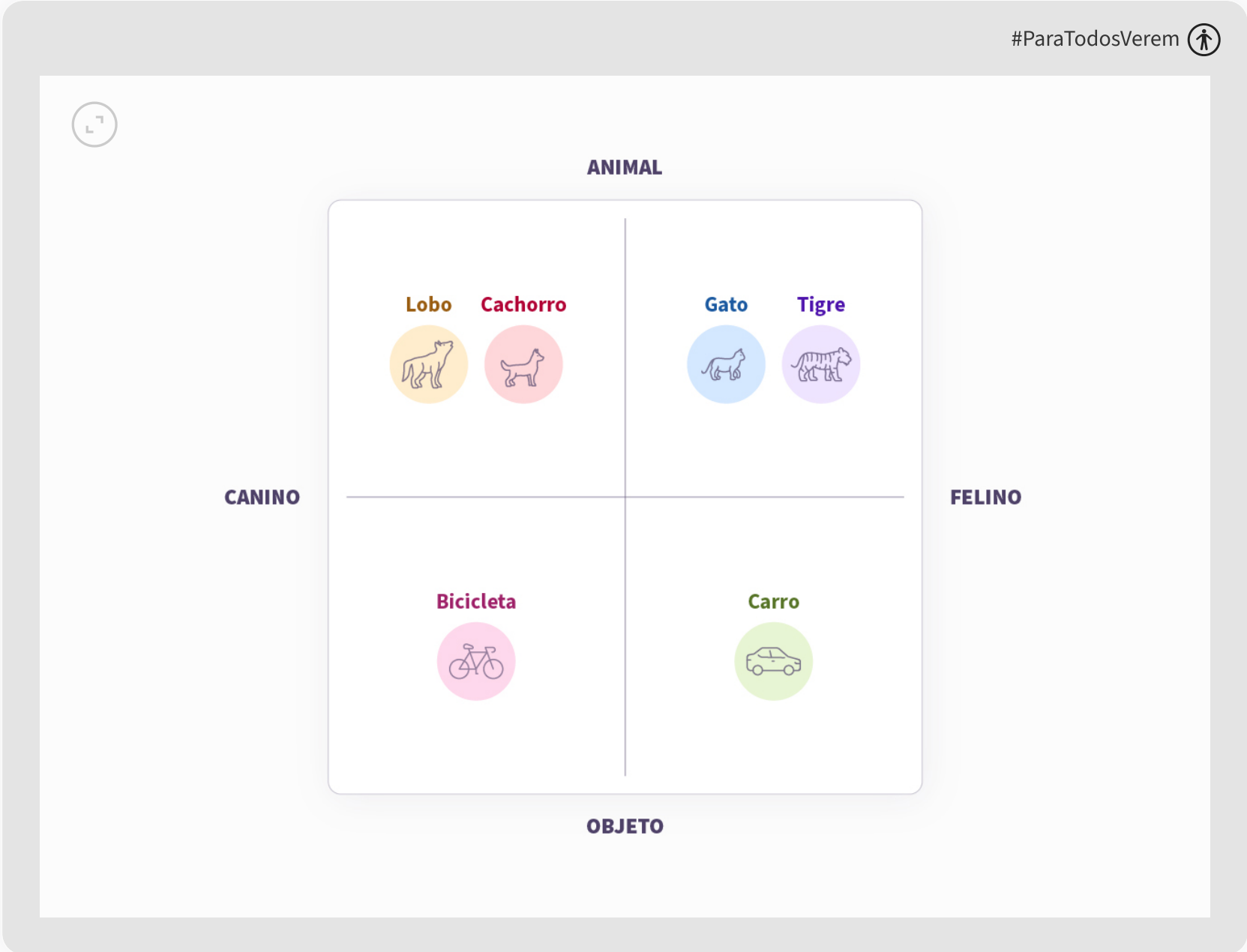
- Eixo X: de "canino" a "felino"
- Eixo Y: de "animal" a "objeto"

#ParaTodosVerem



PUCPR Online.

Agora, sim! O carro fica no quadrante de "objeto", longe dos animais. E podemos adicionar mais coisas:



PUCPR Online.

A bicicleta também é um objeto, mas talvez mais "simples" que um carro. Ela poderia ficar em uma posição ligeiramente diferente, certo?

Mas a vida real é muito mais complexa do que dois eixos. Como representaríamos conceitos como:

- Amor.
- Saudade.
- Pix.
- WhatsApp.
- Quilograma.
- Golfinho.
- Alfândega.
- Copa do Mundo de 2002.
- Aquele cheiro de chuva em terra seca.

Dois eixos não são suficientes. Precisaríamos de eixos para várias outras comparações, por exemplo:

- Concreto *vs.* abstrato.
- Positivo *vs.* negativo (sentimento).
- Tecnológico *vs.* natural.
- Brasileiro *vs.* universal.
- Temporal *vs.* atemporal.

Acho que já pegou a ideia, certo? Pois bem: é exatamente isso que os modelos de *embedding* fazem. Eles representam cada conceito como um vetor de centenas ou milhares de eixos (ou, tecnicamente falando, **dimensões**).

Observe na tabela a seguir que cada modelo de *embedding* utiliza um número diferente de dimensões para representar textos como vetores numéricos.
Essas dimensões funcionam como eixos que capturam características semânticas do conteúdo.
Modelos com mais dimensões conseguem representar significados de forma mais detalhada, porém exigem mais processamento. Já modelos com menos dimensões são mais leves e rápidos, mas com menor nível de nuance.

Modelo	Dimensões
sentence-transformers (all-MiniLM)	384
OpenAI text-embedding-ada-002	1.536
OpenAI text-embedding-3-large	3.072
Google Gecko	768
Cohere embed-v3	1.024

O autor.

CURIOSIDADE

Um humano consegue visualizar facilmente até 3 dimensões (largura, altura, profundidade). Já tentou imaginar 1.536 dimensões? É impossível! Mas matematicamente, funciona perfeitamente. Cada dimensão captura algum aspecto sutil do significado de diferentes conceitos.

O resultado prático funciona bem (afinal, se não funcionasse você provavelmente não teria se interessado por IA em primeiro lugar). Portanto, em um espaço de alta dimensionalidade, palavras com significados parecidos ficam próximas umas das outras:

- "gato" fica perto de "felino", "gatinho", "bichano"
- "gato" fica perto de "cat" (inglês), "Katze" (alemão), "кот" (russo)
- "saudade" fica perto de "nostalgia", "longing", "Sehnsucht"
- "rei" - "homem" + "mulher" ≈ "rainha" (aritmética de vetores)

Essa última propriedade é especialmente boa. Os vetores capturam relações semânticas de forma que você pode fazer "contas" com palavras.

CURIOSIDADE

Pesquisadores descobriram que, nos embeddings do Word2Vec (um modelo antigo), a operação "Paris - França + Itália" resultava em um vetor muito próximo de "Roma". O modelo "aprendeu" que Paris está para França assim como Roma está para Itália, sem nunca ter sido explicitamente ensinado sobre capitais.

Criando *embeddings*

Você deve estar se perguntando: "Tá, mas como o computador aprende a posicionar as palavras corretamente?"

Geralmente fazemos isso durante o treinamento dos modelos de linguagem.

Durante o treinamento (isto é, durante o processo de criação de uma nova versão de uma LLM), ela lê trilhões de frases coletadas da internet, livros, artigos e outros textos. O objetivo dela é um jogo de adivinhação simples: "dada esta sequência de palavras, qual é a próxima palavra mais provável?".

POR EXEMPLO...

"O gato subiu no ___": provavelmente "telhado", "muro", "sofá"

"Bom dia, como você ___": provavelmente "está", "vai"

"A capital do Brasil é ___": provavelmente "Brasília"

E, aí, temos ⬇️

Aprendendo por tentativa e erro

No início do treinamento, o modelo não sabe nada. Ele chuta aleatoriamente. Se a frase é "O gato bebeu ___" e ele chuta "computador", o que está factualmente errado.

O algoritmo de treinamento pune esse erro, forçando o modelo a ajustar seus parâmetros internos.

Com bilhões de exemplos, o modelo começa a perceber padrões:

- "café" e "chá" aparecem frequentemente perto de "xícara", "quente", "beber"
- "cachorro" aparece perto de "latir", "passear", "ração"
- "código" aparece perto de "programar", "bug", "Python"

Para acertar as previsões, o modelo precisa posicionar palavras relacionadas **próximas** no espaço vetorial. Se ele colocar "café" perto de "gasolina", vai errar muitas previsões e será punido. Então ele ajusta, reposiciona e refina até que as palavras fiquem em posições que façam sentido semântico.

O problema disso é que este processo é extremamente caro. Temos trilhões de palavras de texto que requerem milhares de GPUs rodando por semanas ou meses, o que implica no consumo de energia elétrica de uma cidade pequena e um custo de dezenas a centenas de milhões de dólares só para lançar uma nova versão de uma LLM.

FIQUE POR DENTRO

É por isso que as melhores LLMs são treinadas apenas por grandes corporações (OpenAI, Google, Anthropic, Meta).

O processo demanda poder computacional que poucas organizações no mundo possuem. Também é por isso que existem discussões acaloradas sobre propriedade intelectual: esses modelos são treinados em textos da Wikipédia (contribuição comunitária), livros protegidos por copyright, artigos acadêmicos e conteúdo de redes sociais. **Quem é o "dono" do conhecimento que a IA absorveu?**

BUSCA SEMÂNTICA COM *EMBEDDINGS*

Agora que você entende o que são *embeddings*, vamos ver como eles nos ajudam na busca de informações.

Até poucos anos atrás, toda busca em computadores funcionava assim:

1. Você digita algumas palavras.
2. O sistema procura documentos que contenham **exatamente aquelas palavras.**
3. Se as palavras não aparecerem, resultado vazio.

Isso funciona razoavelmente bem quando você sabe exatamente o que procurar. Mas tem sérias limitações. Pensemos em algumas situações:

Problema	Sua busca	Texto no documento	Resultado
Sinônimos	"felino doméstico"	"gatos são ótimos pets"	O termo da sua busca não foi encontrado no texto
Variações de escrita	"trabalho remoto"	"política de *home office*"	
Idiomas diferentes	"*machine learning*"	"aprendizado de máquina"	
Conceitos implícitos	"como economizar dinheiro"	"cortar gastos supérfluos e investir em renda fixa"	

O autor.

Já com os *embeddings*, os resultados mudam completamente. Em vez de procurar por letras, procuramos por significado.

Veja como funciona.

Passo 1: Indexação (feita uma vez)

Cada documento da sua base é convertido em um vetor de *embedding*, onde cada valor representa um daqueles eixos (dimensões) que comentamos anteriormente:

"Gatos são animais domésticos..." → [0.12, -0.45, 0.78, ..., 0.33] (1536 números)

"Política de home office..." → [0.56, 0.23, -0.11, ..., 0.89] (1536 números)

"Receita de bolo de chocolate..." → [-0.34, 0.67, 0.22, ..., -0.15] (1536 números)

Esses vetores são armazenados em um **banco de dados vetorial**.

Passo 2: Busca (feita a cada *query*)

Quando você faz uma pergunta, a sua pergunta é convertida em vetor. Depois, o sistema calcula a **distância** entre esse vetor e todos os vetores armazenados. Os documentos com vetores mais **próximos** são retornados.

POR EXEMPLO...

Query: "felino doméstico" → [0.11, -0.43, 0.75, ..., 0.31]

Documento 1: "Gatos são animais..." → [0.12, -0.45, 0.78, ..., 0.33]

Distância: 0.02

Documento 2: "Receita de bolo..." → [-0.34, 0.67, 0.22, ..., -0.15]

Distância: 0.89

Como o documento 1 tem uma menor distância, é ele que é retornado. Portanto, mesmo que as palavras sejam completamente diferentes ("felino doméstico" x "gatos"), os vetores são próximos porque o significado é similar.

FIQUE POR DENTRO

Na prática, sistemas modernos combinam os dois. Usam busca por palavras-chave para um primeiro filtro rápido, e depois reordenam com busca semântica. Isso dá o melhor dos dois mundos. Essa técnica é chamada de **busca híbrida.**

Medindo "distância" entre conceitos

PRA REFLETIR

Agora, e como calculamos exatamente a "distância" entre dois vetores?

A métrica mais comum é a **similaridade de cosseno.**

Imagine dois vetores como setas saindo da origem de um gráfico. A similaridade de cosseno mede o **ângulo** entre essas setas:

- Se as setas apontam na mesma direção, a similaridade é 1.0 (idênticos).
- Se as setas são perpendiculares, a similaridade é 0.0 (não relacionados).
- Se as setas apontam em direções opostas, a similaridade é -1.0 (opostos).

Na prática, isso significa em cálculos como estes:

Comparação	Similaridade típica
"gato" *vs.* "gato"	1.00 (idêntico)
"gato" *vs.* "felino"	Entre 0.85 e 0.92
"gato" *vs.* "cachorro"	Entre 0.65 e 0.75
"gato" *vs.* "carro"	Entre 0.15 e 0.30
"gato" *vs.* "democracia"	Entre 0.05 e 0.15

O autor.

FIQUE ATENTO

Os valores exatos dependem do modelo de embedding usado.

Modelos diferentes posicionam os conceitos de formas ligeiramente diferentes. Mas a ordem relativa geralmente se mantém: "gato" sempre estará mais próximo de "felino" do que de "carro".

Bancos de dados vetoriais

Agora que entendemos *embeddings* e similaridade, precisamos de um lugar para armazenar milhares ou milhões de vetores e buscar os mais similares de forma eficiente. É aí que entram os bancos de dados vetoriais.

Esses bancos são otimizados para fazer cálculos em vetores, algo que os bancos tradicionais (como o PostgreSQL, MySQL) não são muito bons. Se você tentar fazer busca por similaridade em um banco tradicional com milhões de vetores, vai levar minutos por *query*. **Um banco vetorial faz isso em milissegundos.**

Portanto, para o nosso projeto, usaremos um banco de dados vetorial, como o ChromaDB. Ele roda dentro do Python, consegue gerar *embeddings* automaticamente, e é gratuito. Ele é ótimo para prototipagem.

Naturalmente existem outras opções as quais, a depender do seu interesse e experiência, você pode testar sem problemas. São elas:

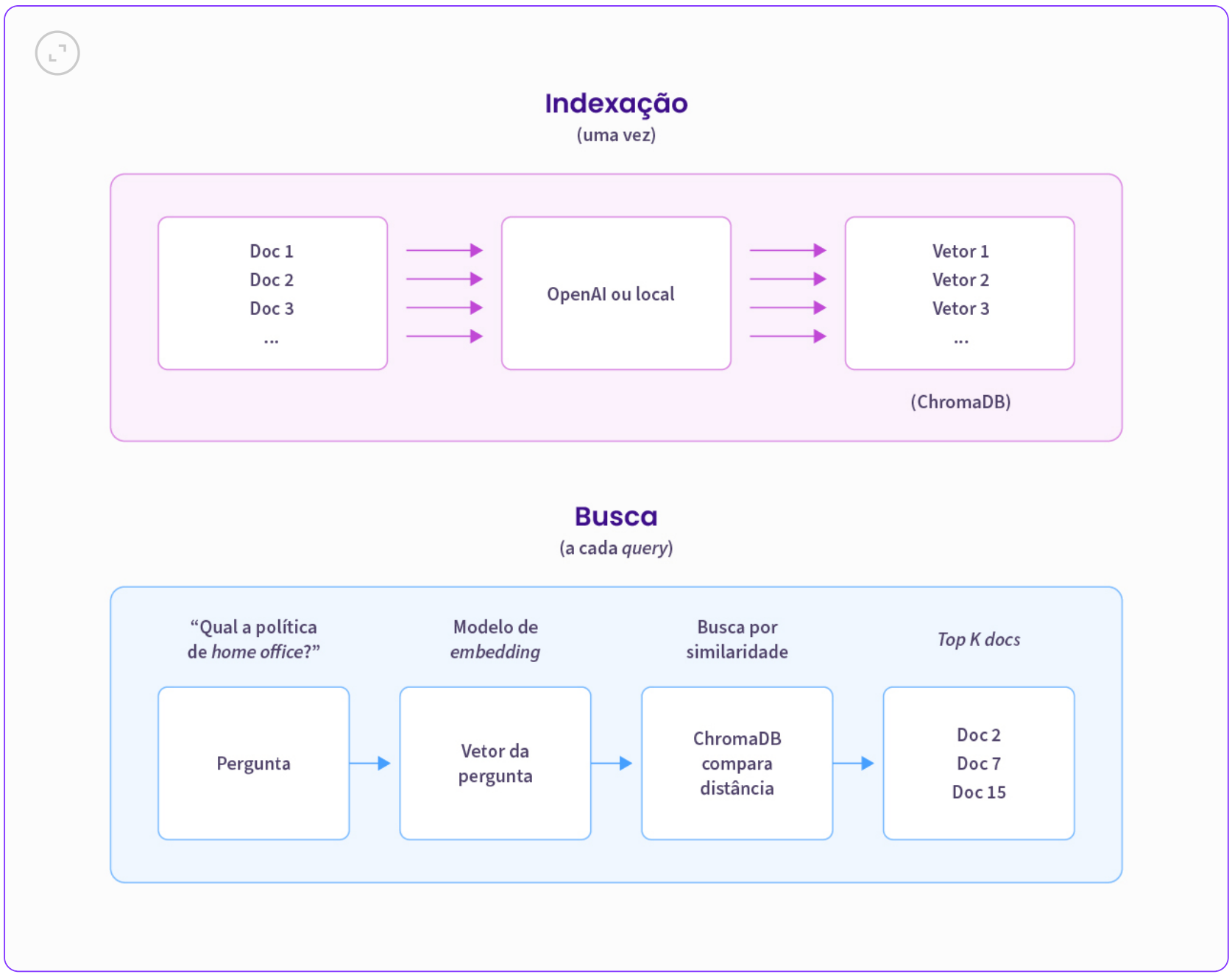
Banco	Características	Quando usar
ChromaDB	Simples, Pythonic	Prototipagem, projetos menores
LanceDB	Integra com DuckDB	Quando já usa DuckDB
Pinecone	Serverless, escalável	Produção em larga escala
Weaviate	Muitas features	Projetos corporativos
Qdrant	Rust, muito rápido	Alta *performance*
Milvus	Distribuído	Bilhões de vetores

O autor.

#FICADICA

Para o nosso projeto, o ChromaDB funciona bem. Se você for para produção com milhões de usuários, considere Pinecone ou similares. Mas os conceitos que você aprende aqui se aplicam a qualquer um deles!

Sendo assim, no final das contas devemos ficar com um funcionamento parecido com este:



breve.

Toda vez que carregamos um novo documento fazemos o processo de indexação. O processo é, basicamente:

1

Carregar documentos: PDFs, TXTs, páginas web etc.

2

Dividir em chunks: documentos longos são divididos em pedaços menores (ex.: parágrafos de 500 palavras)

3

Gerar *embeddings*: cada *chunk* é convertido em vetor

4

Armazenar: vetores são salvos no ChromaDB com metadados (nome do arquivo, página etc.)

Então, a cada pergunta do usuário fazemos o processo de busca:

1

Receber pergunta: aqui, recebemos perguntas como "Qual a política de *home office*?"

2

Gerar *embedding* da pergunta: convertemos a pergunta para vetor

3

Buscar similares: ChromaDB encontra os K vetores mais próximos, onde K é um número pré-estabelecido de vetores (como os 5 ou 10 vetores mais próximos).

4

Retornar documentos: então, os *chunks* originais são retornados, ordenados por relevância

Na próxima aula, combinaremos busca semântica com LLMs para criar um sistema RAG (*retrieval-augmented generation*). Com isso, a LLM responde baseada nos documentos, não inventando informações. Isso resolve o problema das alucinações que discutiremos em breve, já que a LLM deixa de "inventar" e passa a "consultar". É como dar um livro aberto para ela durante a prova.

Mas, primeiro, precisamos dominar a busca semântica. E é isso que faremos na videoaula a seguir.

Para visualizar o vídeo, aponte a câmera do seu celular

Para fechar nosso assunto de hoje, vamos conectar algumas ideias com o *videocase* a seguir. Nele veremos um caso real de busca semântica para a administração pública.

Para visualizar o vídeo, aponte a câmera do seu celular

© PUCPR - Todos os direitos reservados